



TED UNIVERSITY

Faculty of Engineering

Departments of Computer & Software Engineering

Project Name: **DiscoVeR**

Team Members:

CMPE - Levent Bektemur 13759162240

CMPE - Dilge Nisa Üstüntaş 13117245554

SENG - Süzanna Nebioğlu 25123817076

Supervisor:

Ulaş Güleç

Jury Members:

Tansel Dökeroğlu

Tolga Kurtuluş Çapın

Link to the web page of the project:

<https://humanfromnowhere.github.io/discover.github.io/>

1. Introduction

The main objective of this Test Plan is to explain the extent, tactic, schedule and assets of the testing operations for the DiscoVeR Virtual Reality Touring Platform. The aim is to methodically highlight the specifications to be tested, explain the testing methodologies which are applied across the frontend, such as React, backend Express.js, and Supabase, and rendering engine, such as Unity WebGL, and set up the duties and risks related with the evaluation cycle.

2. Roles and Responsibilities

Test Manager / Frontend Tester: Süzanna Nebioğlu– Responsible for Unit Testing the React SPA components, UI/UX validation, and User Acceptance Testing.

Backend Tester: Levent Bektemur– Responsible for testing the Node.js/Express APIs, Supabase database queries, and CORS configuration. And validating JWT authentication, user session security, and data isolation, ensuring users can only modify/delete their own assets via *user_id* checks.

Frontend Developer, Unity & 3D Environment Tester: Dilge Nisa Üstüntaş– Responsible for testing C# coroutines (*ModelYukleyici.cs*), .glb/.gltf model instantiation, material rendering, and camera physics (*Kamera.cs*).

3. Test Environment

Hardware: Standard consumer laptops (Windows/macOS) with a minimum of 8GB RAM, and standard mice for orbital camera testing.

Software/Browsers: Google Chrome and Mozilla Firefox for WebGL compatibility testing.

Backend/Database: Ubuntu VPS hosting Express.js server, Nginx (Reverse Proxy) for HTTPS, and PostgreSQL database for relational data.

Testing Tools: Unity Test Framework (C# scripts), GitHub Issues, bug tracking.

4. Features to be Tested

User Authentication: Login, registration, and session management via Supabase.

Dashboard Navigation: Conditional rendering of Admin vs. User tabs.

File Upload System: Uploading .glb and .gltf files to the Express server's /uploads directory.

3D Rendering Engine: The ability of the model loader to fetch binary (.glb) and JSON-based (.gltf) data from the server, instantiate the prefabs, and correctly apply embedded textures/materials.

User Interaction: Camera orbital navigation (zoom, pan, rotate) within the Unity WebGL viewport.

System Integration: The Unity Messaging API (SendMessage) bridges JavaScript commands to Unity prefabs.

Data Isolation & Security: JWT-based route protection verifying that users can only fetch (GET) and delete (DELETE) models directly tied to their own unique user_id.

5. Testing Methodology

Unit Testing: We will test functions and individual classes in isolation. For example, testing the Express server's file validation to ensure no other format is accepted but .glb and .gltf extensions.

Integration Testing: Ensuring the React frontend correctly communicates with the Express backend, and the JavaScript *viewer.js* successfully passes messages to the Unity WebGL build.

System Testing: End-to-end testing of the fully compiled platform to ensure all components work together as a whole, without any issues.

Performance Testing: We'll load high-poly 3D models to measure rendering times, frame rate (FPS) stability in WebGL, and memory consumption during .glb payload extraction.

User Acceptance Testing (UAT): End-users will navigate the platform to validate that the UI is intuitive, comfortable and easy to use and the 3D touring features meet user expectations.

Beta Testing: A limited release to a small group of our classmates to identify any edge-case bugs before the final presentation to jury members.

6. Test Cases

Test ID	Feature	Description	Pre-conditions	Expected Result	Pass/Fail
TC-01	UI Routing	User switches between "Dashboard" and "Models" tabs.	User is logged in.	The view updates instantly via SPA logic without page reload.	[FAIL]
TC-02	API / Backend	Admin uploads a standard .glb or .gltf file.	File size is under 50MB.	Server responds with 200 OK and file is saved in /uploads.	[PASS]
TC-03	Integration	viewer.js triggers ModelAc('test.glb') .	Unity WebGL is initialized.	SendMessage successfully calls Yonetici prefab in Unity.	[PASS]
TC-04	Unity Rendering	Model loader fetches a .glb file.	Valid .glb URL provided.	Model is successfully instantiated into the scene with materials/textures intact.	[FAIL]
TC-05	Unity Physics	User drags the mouse in the WebGL viewport.	A 3D model is loaded.	Kamera.cs processes input and orbits the camera smoothly.	[PASS]

TC-06	Performance	Load a heavy .glb model (e.g., 100k+ polygons).	System is idle.	Model loads within 5 seconds; WebGL does not crash the browser.	[FAIL]
TC-07	Auth / API	User attempts to fetch models without a valid JWT token.	User is logged out or token is expired.	Server responds with <i>401 Unauthorized</i> and no data is sent.	[PASS]
TC-08	Data Isolation	User views their dashboard to list models.	User is logged in with valid token.	Only models matching the user's <i>user_id</i> are fetched from PostgreSQL and displayed.	[PASS]
TC-09	Security / Delete	User attempts to delete a model belonging to another user.	User is logged in, but targets a foreign model ID.	Server responds with <i>403 Forbidden</i> ; the file and DB record remain intact.	[FAIL]

7. Test Schedule & Control Procedures

- **Schedule:**
 - Unit Testing: Week 10
 - Integration & System Testing: Week 11 - 12
 - Performance Testing: Week 13
 - UAT & Beta Testing: Week 14
- **Control Procedures:** All bugs we may identify during the testing period we will log as Issues in the project's GitHub repository. Severity level ; Low, Medium, High, Critical will be assigned to each issue. Before the branch is merged into the main production branch each bug has to be resolved and re-tested.

8. Risks and Mitigation

Risk	Impact	Mitigation Strategy
WebGL Browser Crashes: Unoptimized .glb models may exceed browser memory limits.	High	Implement file size limits on uploads and encourage administrators to use Draco compression for .glb assets.
CORS Policy Blocking: API requests between React and Node.js fail.	High	Ensure the Express cors middleware is configured correctly for both local testing and production domains.
Texture/Material Failures: The .gltf file loads but appears entirely pink or untextured in WebGL.	Medium	Ensure that .gltf files are uploaded alongside their associated .bin and texture files, or enforce a strict .glb (single binary) only policy for easier handling.
Unauthorized Data Manipulation: Malicious users attempting to delete or overwrite models belonging to others via API requests.	Critical	Enforce strict JWT validation on the Express router. Every DELETE request must verify that the requested model's user_id strictly matches the token's payload before querying PostgreSQL